

Package ‘drcmd’

April 28, 2026

Title Doubly-Robust Causal Inference with Missing Data

Version 0.0.0.9000

Description Doubly-robust estimation of counterfactual means in the presence of missing data. The `drcmd()` function estimates counterfactual means for binary point treatments and reports average treatment effects, as well as causal risk ratios and odds ratios for binary outcomes. General missingness patterns in the data are allowed and automatically determined by the function -- the only requirement is that any missingness occurs at random conditional on variables that are always available. For scenarios where non-missingness probabilities are known, as is common in two-phase sampling designs, users can provide the non-missingness probabilities through the `Rprobs` argument. Users can fit nuisance functions through either highly-adaptive LASSO (HAL) or SuperLearner, the latter of which the user must specify libraries.

License GPL (>= 3)

URL <https://github.com/keithbarnatchez/drcmd>

BugReports <https://github.com/keithbarnatchez/drcmd/issues>

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports SuperLearner, ggplot2

Suggests gam, hal9001, nnls, testthat (>= 3.0.0)

Config/testthat/edition 3

NeedsCompilation no

Author Keith Barnatchez [aut, cre] (ORCID:
<<https://orcid.org/0000-0002-1451-9335>>),
Griffin DesRoches [aut]

Maintainer Keith Barnatchez <keithbarnatchez@gmail.com>

Contents

<code>check_binary</code>	2
<code>check_entry_errors</code>	3
<code>check_r_ind</code>	3
<code>clean_learners</code>	4
<code>create_folds</code>	5

drcmd	6
drcmd_est	8
drcmd_est_fold	9
est_g	11
est_kappa	12
est_m_a	13
est_psi	14
est_psi_tml	15
est_ses_crossfit	16
est_varphi	17
est_varphi_eem	17
est_varphi_main	18
find_missing_pattern	20
get_nuisance_ests	21
get_phi_hat	22
get_sl_libraries	23
plot.drcmd	23
predict.SL.hal9001	24
print.drcmd	25
SL.hal9001	26
summary.drcmd	26
tml_updates	27
trim	28
truncate_g	29
truncate_r	29

check_binary

check_binary

Description

Check if the outcome variable is 0/1 binary. Return 1 if true, 0 if false

Usage

```
check_binary(x)
```

Arguments

x A numeric vector

Value

A logical value

check_entry_errors	<i>Check arguments to drcmd for entry errors</i>
--------------------	--

Description

Checks if the arguments to main function drcmd are correctly specified. Returns TRUE if all checks are passed, throws an error with message otherwise.

Usage

```
check_entry_errors(Y, A, X, W, R, eem_ind, Rprobs, k)
```

Arguments

Y	A vector or data frame containing outcome values
A	A vector or data frame containing treatment variable values
X	A data frame containing covariate values
W	A data frame containing proxy variable values
R	A vector containing missingness indicator variable

Examples

```
## Not run:
n <- 200
X <- data.frame(X1 = rnorm(n))
W <- data.frame(W1 = rnorm(n))
A <- rbinom(n, 1, 0.5)
Y <- rnorm(n)
R <- rbinom(n, 1, 0.7)
check_entry_errors(Y, A, X, W, R, eem_ind = FALSE, Rprobs = NA, k = 1)

## End(Not run)
```

check_r_ind	<i>check_r_ind</i>
-------------	--------------------

Description

Check if the missingness indicator R is defined correctly

Usage

```
check_r_ind(data, Y, A, X, W, R)
```

Arguments

data	A data frame
Y	A character string containing outcome variable name
A	A character string containing treatment variable name
X	A character vector containing covariate variable names
W	A character vector containing weight variable names
R	A character string containing randomization variable name

Value

A logical value

Examples

```
## Not run:
n <- 200
R <- rbinom(n, 1, 0.7)
data <- data.frame(
  Y = ifelse(R == 1, rnorm(n), NA),
  A = ifelse(R == 1, rbinom(n, 1, 0.5), NA),
  X1 = rnorm(n),
  R = R
)
check_r_ind(data, Y = "Y", A = "A", X = "X1", W = character(0), R = "R")

## End(Not run)
```

clean_learners

Clean SuperLearner libraries

Description

Internal function for setting SuperLearner libraries before they are passed into estimation procedures. Default libraries from the `sl_learners` argument are used to fill in missing libraries for any nuisance function with libraries left unspecified

Usage

```
clean_learners(
  default_learners,
  m_learners,
  g_learners,
  r_learners,
  po_learners
)
```

Arguments

default_learners	Either null, or a character vector containing SuperLearner libraries to use for estimating all nuisance functions. User can alternatively specify libraries for each nuisance function for added flexibility
m_learners	Either null, or a character vector containing SuperLearner libraries to be used for the outcome regression
g_learners	Either null, or a character vector containing SuperLearner libraries to be used for the propensity scores
r_learners	Either null, or a character vector containing SuperLearner libraries to be used for the missingness indicator regression
po_learners	Either null, or a character vector containing SuperLearner libraries to be used for the pseudo outcome regression

Value

A list of

create_folds	<i>Create folds for cross-fitting</i>
--------------	---------------------------------------

Description

Given length of data, creates k folds and returns a list of all possible train-test pairs

Usage

```
create_folds(n, k)
```

Arguments

n	Length of data
k	Number of desired folds

Value

A list of train-test pairs

drcmd

*Doubly-robust causal inference with missing data***Description**

Doubly-robust estimation of counterfactual means in the presence of missing data. The `drcmd()` function estimates counterfactual means for binary point treatments and reports average treatment effects, as well as causal risk ratios and odds ratios for binary outcomes. General missingness patterns in the data are allowed and automatically determined by the function – the only requirement is that any missingness occurs at random conditional on variables that are always available. For scenarios where non-missingness probabilities are known, as is common in two-phase sampling designs, users can provide the non-missingness probabilities through the `Rprobs` argument. Users can fit nuisance functions through either highly-adaptive LASSO (HAL) or SuperLearner, the latter of which the user must specify libraries.

Usage

```
drcmd(
  Y,
  A,
  X,
  W = NA,
  R = NA,
  default_learners = NULL,
  m_learners = NULL,
  g_learners = NULL,
  r_learners = NULL,
  po_learners = NULL,
  eem_ind = FALSE,
  tml = FALSE,
  Rprobs = NA,
  k = 1,
  cutoff = 0.025,
  quiet = TRUE,
  cv_folds = 5,
  parallel = FALSE,
  att = FALSE,
  atc = FALSE
)
```

Arguments

Y	Outcome variable. Can be continuous or binary
A	A binary treatment variable (1=treated, 0=control)
X	Dataframe containing baseline covariates
W	(optional) Dataframe containing variables solely predictive of missingness, but not a cause of the outcome or exposure.
R	(optional) A character string specifying the missingness indicator, where 0 indicates missing data. If not specified, the function will create the missingness indicator by identifying the missingness pattern in the data

default_learners	A character vector containing SuperLearner libraries to use for estimating all nuisance functions. User can alternatively specify libraries for each nuisance function for added flexibility
m_learners	A character vector containing learners to be used for the outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
g_learners	A character vector containing learners to be used for the propensity score. User can specify 'hal' or a vector of SuperLearner libraries
r_learners	A character vector containing learners to be used for the missingness indicator regression. User can specify 'hal' or a vector of SuperLearner libraries
po_learners	A character vector containing learners to be used for the pseudo-outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
eem_ind	A logical indicating whether to use empirical efficiency maximization
tml	A logical indicating whether to obtain estimates through targeted maximum likelihood estimation (TML). If TRUE, estimates obtained by TML. If FALSE (default setting), estimates are obtained through a one-step estimator
Rprobs	A vector of probabilities for the missingness indicator. Only suitable for study designs where researcher controls mechanism by which variables are missing (e.g. two-phase sample designs). Defaults to NA, in which case missingness probabilities are estimated.
k	A numeric indicating the number of folds for cross-fitting
cutoff	Cutoff for treatment and complete case propensity scores. Estimates outside of [cutoff, 1-cutoff] are set to cutoff or 1-cutoff, respectively
quiet	Logical indicating whether to suppress progress messages. Default is TRUE. Set to FALSE to see which nuisance function is being estimated.
cv_folds	Number of cross-validation folds used internally by SuperLearner for model selection. Default is 5. Lower values speed up estimation.
parallel	Logical indicating whether to run cross-fitting folds in parallel using <code>parallel::mclapply</code> . Only used when $k > 1$. Note: not supported on Windows.
att	Logical indicating whether to additionally estimate the average treatment effect on the treated (ATT). Default is FALSE.
atc	Logical indicating whether to additionally estimate the average treatment effect on the controls (ATC). Default is FALSE.

Value

An S3 object of class "drcmd" containing estimation results, information on the missing data structure, and parameters used in the estimation

params A list containing parameters used in the estimation

Z A character vector containing always-available variables

R A character vector containing the complete case indicator values

U A character vector containing partially-missing variables

results A list of dataframes storing (i) point estimates, (ii) standard errors, and (iii) nuisance function estimates

Examples

```

set.seed(1)
n <- 200
X <- rnorm(n)
A <- rbinom(n, 1, plogis(X))
Y <- rnorm(n) + A + X
R_ind <- rbinom(n, 1, plogis(X))
Y[R_ind == 0] <- NA
covariates <- data.frame(X = X)

fit <- drcmd(Y, A, covariates,
             default_learners = "SL.glm",
             k = 1)

fit

```

drcmd_est	<i>Obtain doubly-robust counterfactual mean estimates through cross-fitting</i>
-----------	---

Description

Outer function for estimating counterfactual means through cross-fitting. Calls `drcmd_est_fold` to obtain estimates for each cross-fitting fold

Usage

```

drcmd_est(
  Y,
  A,
  X,
  Z,
  R,
  m_learners,
  g_learners,
  r_learners,
  po_learners,
  eem_ind,
  tml,
  Rprobs,
  k,
  cutoff,
  y_bin,
  yscaled,
  cv_folds = 5,
  parallel = FALSE,
  quiet = TRUE,
  att = FALSE,
  atc = FALSE
)

```

Arguments

Y	Outcome variable. Can be continuous or binary
A	A binary treatment variable (1=treated, 0=control)
X	Dataframe containing baseline covariates
Z	Dataframe containing all variables that are never subject to missingness
R	Binary missingness indicator
m_learners	A character vector containing learners to be used for the outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
g_learners	A character vector containing learners to be used for the propensity score. User can specify 'hal' or a vector of SuperLearner libraries
r_learners	A character vector containing learners to be used for the missingness indicator regression. User can specify 'hal' or a vector of SuperLearner libraries
po_learners	A character vector containing learners to be used for the pseudo-outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
eem_ind	A logical indicating whether to use empirical efficiency maximization
Rprobs	A vector of probabilities for R
k	A numeric indicating the number of folds for cross-fitting

Value

A list of point estimates and standard errors for all estimands considered

Examples

```
## Not run:
n <- 500
X <- data.frame(X1 = rnorm(n))
A <- rbinom(n, 1, plogis(X$X1))
Y <- A + X$X1 + rnorm(n)
R <- rbinom(n, 1, plogis(X$X1))
Y[R == 0] <- 0; A[R == 0] <- 0
Z <- X
res <- drcmd_est(Y, A, X, Z, R,
  m_learners = "SL.glm", g_learners = "SL.glm",
  r_learners = "SL.glm", po_learners = "SL.glm",
  eem_ind = FALSE, tml = FALSE,
  Rprobs = NA, k = 1, cutoff = 0.025,
  y_bin = FALSE, yscaled = FALSE)

## End(Not run)
```

drcmd_est_fold

Calculate point estimates within a single cross-fitting fold

Description

Outer function for obtaining point estimates for single fold

Usage

```
drcmd_est_fold(
  splits,
  Y,
  A,
  X,
  Z,
  R,
  m_learners,
  g_learners,
  r_learners,
  po_learners,
  eem_ind,
  tml,
  Rprobs,
  cutoff,
  y_bin,
  cv_folds = 5,
  quiet = TRUE,
  att = FALSE,
  atc = FALSE
)
```

Arguments

<code>splits</code>	A list of train/test indices
<code>Y</code>	Outcome variable. Can be continuous or binary
<code>A</code>	A binary treatment variable (1=treated, 0=control)
<code>X</code>	Dataframe containing baseline covariates
<code>Z</code>	Dataframe containing all variables that are never subject to missingness
<code>R</code>	Binary missingness indicator
<code>m_learners</code>	A character vector containing learners to be used for the outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
<code>g_learners</code>	A character vector containing learners to be used for the propensity score. User can specify 'hal' or a vector of SuperLearner libraries
<code>r_learners</code>	A character vector containing learners to be used for the missingness indicator regression. User can specify 'hal' or a vector of SuperLearner libraries
<code>po_learners</code>	A character vector containing learners to be used for the pseudo-outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
<code>eem_ind</code>	A logical indicating whether to use empirical efficiency maximization
<code>Rprobs</code>	A vector of probabilities for R

Value

A list of results from estimation on current fold

Examples

```
## Not run:
n <- 500
X <- data.frame(X1 = rnorm(n))
A <- rbinom(n, 1, plogis(X$X1))
Y <- A + X$X1 + rnorm(n)
R <- rbinom(n, 1, plogis(X$X1))
Y[R == 0] <- 0; A[R == 0] <- 0
Z <- X
splits <- list(train = 1:n, test = 1:n)
fold_res <- drcmd_est_fold(splits, Y, A, X, Z, R,
  m_learners = "SL.glm", g_learners = "SL.glm",
  r_learners = "SL.glm", po_learners = "SL.glm",
  eem_ind = FALSE, tml = FALSE,
  Rprobs = NA, cutoff = 0.025, y_bin = FALSE)

## End(Not run)
```

est_g

Estimate the propensity score

Description

Function for obtaining propensity score estimates $P(A=a|X)$

Usage

```
est_g(idx, A, X, R, kappa_hat, g_learners, cv_folds = 5)
```

Arguments

idx	Indices to carry out estimation over
A	A binary treatment variable (1=treated, 0=control)
X	Dataframe containing baseline covariates
R	Binary missingness indicator, where 0 indicates missing data
kappa_hat	A numeric vector containing the fitted values of kappa
g_learners	A character vector containing the names of the learners for estimation
cv_folds	Number of cross-validation folds for SuperLearner

Value

A list containing the estimate of $E[Y \mid A=a, X, W]$

Examples

```
## Not run:
n <- 500
X <- rnorm(n)
A <- rbinom(n, 1, plogis(X))
R <- rbinom(n, 1, plogis(X))
A[R == 0] <- 0
```

```

X <- data.frame(X)
kappa_hat <- rep(1, n)
g_learners <- c('SL.glm')
g_hat <- est_g(idx = 1:n, A = A, X = X, R = R,
               kappa_hat = kappa_hat, g_learners = g_learners)

## End(Not run)

```

est_kappa

Estimate complete case propensity scores

Description

Function for obtaining complete case propensity score estimates

Usage

```
est_kappa(idx, Z, R, r_learners, cv_folds = 5)
```

Arguments

idx	Indices to carry out estimation over
Z	Dataframe containing all non-missing variables
R	Binary missingness indicator, where 0 indicates missing data
r_learners	A character vector specifying learners to be used for estimation

Value

A numeric vector containing the estimate of $E[R \mid Z]$

Examples

```

## Not run:
n <- 500
Z <- rnorm(n)
R <- rbinom(n, 1, plogis(Z))
Z <- data.frame(Z)
r_learners <- c('SL.glm')
kappa_hat <- est_kappa(idx = 1:n, Z = Z, R = R,
                       r_learners = r_learners)

## End(Not run)

```

est_m_a	<i>Estimating outcome regression</i>
---------	--------------------------------------

Description

Function for obtaining estimate of $E[Y \mid A=a, X]$. Estimation is carried out with SuperLearner, using learners specified in `m_learners`

Usage

```
est_m_a(idx, Y, A, X, R, kappa_hat, m_learners, cv_folds = 5)
```

Arguments

<code>idx</code>	Indices to carry out estimation over
<code>Y</code>	Outcome variable. Can be continuous or binary
<code>A</code>	A binary treatment variable (1=treated, 0=control)
<code>X</code>	Dataframe containing baseline covariates
<code>R</code>	Binary missingness indicator, where 0 indicates missing data
<code>kappa_hat</code>	A numeric vector containing the fitted values of kappa
<code>m_learners</code>	A character vector containing the names of the superlearner algorithms

Value

A list containing the estimate of $E[Y \mid A=a, X]$

Examples

```
## Not run:
n <- 500
X <- rnorm(n)
A <- rbinom(n, 1, plogis(X))
R <- rbinom(n, 1, plogis(X))
Y <- A + X + rnorm(n)
Y[R == 0] <- 0; A[R == 0] <- 0
X <- data.frame(X)
kappa_hat <- rep(1, n)
m_learners <- c('SL.glm')
m_hat <- est_m_a(idx = 1:n, Y = Y, A = A, X = X, R = R,
                 kappa_hat = kappa_hat, m_learners = m_learners)

## End(Not run)
```

est_psi

*Obtain estimates for current cross fitting fold***Description**

Function for obtaining counterfactual mean estimates for the current fold of the cross-fitting procedure

Usage

```
est_psi(
  idx,
  R,
  Z,
  kappa_hat,
  phi_hat,
  varphi_hat,
  y_bin,
  att = FALSE,
  atc = FALSE,
  plugin_att = NULL,
  plugin_atc = NULL,
  phi_att_hat = NULL,
  phi_atc_hat = NULL
)
```

Arguments

idx	A data frame
R	A character string containing randomization variable name
Z	A character vector containing the names of the variables in Z
kappa_hat	A numeric vector containing the fitted values of kappa
phi_hat	A list containing the estimate of $E[\phi \mid Z]$
varphi_hat	A list containing the estimate of $E[\phi \mid Z]$

Value

A list of point estimates and standard errors for counterfactual means and various counterfactual contrasts

Examples

```
## Not run:
# Typically called internally by drcmd_est_fold() after obtaining
# nuisance estimates, phi_hat, and varphi_hat.

## End(Not run)
```

est_psi_tml

*Estimate causal effects through targeted maximum likelihood***Description**

Function for obtaining estimates of causal estimands through targeted maximum likelihood. To be called within `drcmd_est_fold()`. Updates estimate of $P(R=1|Z)$ and the plug-in estimator in a manner which removes the plug-in bias incurred by an initial plug-in estimator

Usage

```
est_psi_tml(
  idx,
  Y,
  A,
  X,
  R,
  Z,
  m_1_hat,
  m_0_hat,
  g_hat,
  kappa_hat,
  phi_1_hat,
  phi_0_hat,
  varphi_hat,
  maxits = 10,
  att = FALSE,
  atc = FALSE
)
```

Arguments

<code>idx</code>	Indices of the training set
<code>Y</code>	Outcome vector
<code>A</code>	Treatment vector
<code>X</code>	Dataframe of covariates
<code>R</code>	Complete case indicator vector
<code>Z</code>	Dataframe containing all variables that are never subject to missingness
<code>m_1_hat</code>	A numeric vector containing the fitted outcome predictions under $A=1$
<code>m_0_hat</code>	A numeric vector containing the fitted outcome predictions under $A=0$
<code>g_hat</code>	A numeric vector containing the fitted treatment propensity scores
<code>kappa_hat</code>	A numeric vector containing the fitted values of κ
<code>phi_1_hat</code>	Estimates of EIC for $E[Y(1)]$
<code>phi_0_hat</code>	Estimates of $E[Y(0)]$
<code>varphi_hat</code>	A list containing estimate of $E[\phi_a Z]$

Value

A list containing updated estimates of the nuisance parameters

Examples

```
## Not run:
# Typically called internally by drcmd_est_fold() when tml = TRUE.
# Requires nuisance estimates from get_nuisance_ests() and
# pseudo-outcome estimates from est_varphi_main().

## End(Not run)
```

est_ses_crossfit	<i>Cross-fit SE estimation via influence curve contributions</i>
------------------	--

Description

Estimates cross-fit SEs by taking contributions to influence curve of each fold

Usage

```
est_ses_crossfit(res, y_bin, att = FALSE, atc = FALSE)
```

Arguments

res	A list of per-fold results from drcmd_est_fold, each containing an ics data frame of influence curve contributions
y_bin	Logical indicating whether the outcome is binary
att	Logical indicating whether to compute SEs for the ATT
atc	Logical indicating whether to compute SEs for the ATC

Value

A list of cross-fit SEs

Examples

```
## Not run:
# Typically called internally by drcmd_est() after cross-fitting.
# res is a list of per-fold results from drcmd_est_fold(), each
# containing an $ics data frame of influence curve contributions.

## End(Not run)
```

est_varphi	<i>Perform pseudo-outcome regression with conventional loss function</i>
------------	--

Description

Outer function for obtaining estimate of $E[\phi \mid Z]$

Usage

```
est_varphi(idx, R, Z, phi_1_hat, phi_0_hat, po_learners, Y, cv_folds = 5)
```

Arguments

idx	Indices to carry out estimation over
R	Binary missingness indicator, where 0 indicates missing data
Z	Dataframe containing all non-missing variables
phi_1_hat	A numeric vector containing the fitted values of ϕ under $A=1$
phi_0_hat	A numeric vector containing the fitted values of ϕ under $A=0$
po_learners	A character vector containing the names of the superlearner algorithms

Value

A list containing the estimate of $E[\phi \mid Z]$ for $a=0$ and $a=1$

Examples

```
## Not run:
n <- 500
Z <- data.frame(Z1 = rnorm(n))
R <- rbinom(n, 1, 0.7)
phi_1_hat <- rnorm(n)
phi_0_hat <- rnorm(n)
Y <- rnorm(n)
varphi <- est_varphi(1:n, R, Z, phi_1_hat, phi_0_hat,
                    po_learners = "SL.glm", Y = Y)

## End(Not run)
```

est_varphi_eem	<i>Perform pseudo-outcome regression with empirical efficiency maximization</i>
----------------	---

Description

Function for obtaining estimate of $E[\phi_a \mid Z]$ via empirical efficiency maximization

Usage

```
est_varphi_eem(
  idx,
  R,
  Z,
  phi_1_hat,
  phi_0_hat,
  kappa_hat,
  po_learners,
  Y,
  cv_folds = 5
)
```

Arguments

idx	Indices to carry out estimation over
R	Binary missingness indicator, where 0 indicates missing data
Z	Dataframe containing all non-missing variables
phi_1_hat	Vector of predicted phi under A=1
phi_0_hat	Vector of predicted phi under A=0
kappa_hat	A numeric vector containing the fitted values of kappa
po_learners	A character vector containing the names of the superlearner algorithms
Y	Outcome variable. Can be continuous or binary

Value

A list containing the estimate of $E[\phi_a | Z]$ for $a=0$ and $a=1$

Examples

```
## Not run:
n <- 500
Z <- data.frame(Z1 = rnorm(n))
R <- rbinom(n, 1, 0.7)
phi_1_hat <- rnorm(n)
phi_0_hat <- rnorm(n)
kappa_hat <- rep(0.7, n)
Y <- rnorm(n)
varphi <- est_varphi_eem(1:n, R, Z, phi_1_hat, phi_0_hat,
                        kappa_hat, po_learners = "SL.glm", Y = Y)

## End(Not run)
```

est_varphi_main

Perform pseudo-outcome regression

Description

Function for obtaining estimate of $E[\phi_a | Z]$ through pseudo-outcome regression. Calls inner function to perform estimation, depending on whether user wishes to perform empirical efficiency maximization or not

Usage

```
est_varphi_main(
  idx,
  R,
  Z,
  phi_1_hat,
  phi_0_hat,
  kappa_hat,
  eem_ind,
  po_learners,
  Y,
  cv_folds = 5,
  quiet = TRUE,
  att = FALSE,
  atc = FALSE,
  phi_att_hat = NULL,
  phi_atc_hat = NULL
)
```

Arguments

idx	Indices to carry out estimation over
R	Binary missingness indicator, where 0 indicates missing data
Z	Dataframe containing all non-missing variables
phi_1_hat	Vector of pseudo-outcome values under A=1
phi_0_hat	Vector of pseudo-outcome values under A=0
kappa_hat	A numeric vector containing the fitted values of kappa
eem_ind	A logical value indicating whether to estimate via EEM
po_learners	A character vector containing the names of the SuperLearner algorithms
Y	Outcome variable. Can be continuous or binary
cv_folds	Number of cross-validation folds for SuperLearner
quiet	Logical indicating whether to suppress progress messages
att	Logical indicating whether to compute pseudo-outcome regression for ATT
atc	Logical indicating whether to compute pseudo-outcome regression for ATC

Value

A list containing the estimate of $E[\phi_a | Z]$ for $a=0$ and $a=1$

Examples

```
## Not run:
n <- 500
Z <- data.frame(Z1 = rnorm(n))
R <- rbinom(n, 1, 0.7)
phi_1_hat <- rnorm(n)
phi_0_hat <- rnorm(n)
kappa_hat <- rep(0.7, n)
Y <- rnorm(n)
varphi <- est_varphi_main(1:n, R, Z, phi_1_hat, phi_0_hat,
```

```

kappa_hat, eem_ind = FALSE,
po_learners = "SL.glm", Y = Y)

## End(Not run)

```

```
find_missing_pattern  find_missing_pattern
```

Description

Find the missing pattern in the data

Usage

```
find_missing_pattern(Y, A, X, W)
```

Arguments

Y	A vector or data frame containing outcome values
A	A vector or data frame containing treatment variable values
X	A data frame containing covariate values
W	A data frame containing proxy variable values

Value

A character string containing the missing pattern

Examples

```

## Not run:
n <- 200
X <- data.frame(X1 = rnorm(n))
A <- rbinom(n, 1, 0.5)
Y <- rnorm(n)
R <- rbinom(n, 1, 0.7)
Y[R == 0] <- NA
W <- X[, 0]
result <- find_missing_pattern(Y, A, X, W)
result$Z # variables without missingness
result$U # variables with missingness
result$R # complete case indicator

## End(Not run)

```

get_nuisance_ests	<i>Obtain nuisance function estimates</i>
-------------------	---

Description

Function for obtaining estimates of all relevant nuisance functions

Usage

```
get_nuisance_ests(
  idx,
  Y,
  A,
  X,
  Z,
  R,
  m_learners,
  g_learners,
  r_learners,
  Rprobs,
  cutoff,
  cv_folds = 5,
  quiet = TRUE
)
```

Arguments

idx	Indices to carry out estimation over
Y	Outcome variable. Can be continuous or binary
A	A binary treatment variable (1=treated, 0=control)
X	Dataframe containing baseline covariates
Z	Dataframe containing all non-missing variables
R	Binary missingness indicator, where 0 indicates missing data
m_learners	A character vector containing learners to be used for the outcome regression. User can specify 'hal' or a vector of SuperLearner libraries
g_learners	A character vector containing learners to be used for the propensity score. User can specify 'hal' or a vector of SuperLearner libraries
r_learners	A character vector containing learners to be used for the missingness indicator regression. User can specify 'hal' or a vector of SuperLearner libraries
Rprobs	A vector of probabilities for R
cutoff	Cutoff for propensity score truncation
cv_folds	Number of cross-validation folds for SuperLearner
quiet	Logical indicating whether to suppress progress messages

Value

A list of nuisance estimates

Examples

```
## Not run:
n <- 500
X <- data.frame(X1 = rnorm(n))
A <- rbinom(n, 1, plogis(X$X1))
Y <- A + X$X1 + rnorm(n)
R <- rbinom(n, 1, plogis(X$X1))
Y[R == 0] <- 0; A[R == 0] <- 0
Z <- X
nuis <- get_nuisance_ests(1:n, Y, A, X, Z, R,
                        m_learners = "SL.glm",
                        g_learners = "SL.glm",
                        r_learners = "SL.glm",
                        Rprobs = NA, cutoff = 0.025)

## End(Not run)
```

get_phi_hat

*Construct full data EIF estimate***Description**

Function for constructing estimate of the full data EIF, given propensity score and outcome model fits

Usage

```
get_phi_hat(Y, A, X, R, Z, g_hat, m_a_hat, kappa_hat)
```

Arguments

Y	Outcome variable. Can be continuous or binary
A	A binary treatment variable (1=treated, 0=control)
X	Dataframe containing baseline covariates
R	Binary missingness indicator
Z	Dataframe containing all variables that are never subject to missingness
g_hat	Propensity score predictions
m_a_hat	List of outcome predictions under A=0/1
kappa_hat	Missingness probabilities

Examples

```
## Not run:
n <- 500
X <- data.frame(X1 = rnorm(n))
A <- rbinom(n, 1, plogis(X$X1))
Y <- A + X$X1 + rnorm(n)
R <- rep(1, n)
Z <- X
g_hat <- plogis(X$X1)
m_a_hat <- list(m_1_hat = 1 + X$X1, m_0_hat = X$X1)
```

```
kappa_hat <- rep(1, n)
phi <- get_phi_hat(Y, A, X, R, Z, g_hat, m_a_hat, kappa_hat)

## End(Not run)
```

get_sl_libraries	<i>List SuperLearner libraries</i>
------------------	------------------------------------

Description

List all available SuperLearner libraries

Usage

```
get_sl_libraries()
```

Value

A character vector of all available SuperLearner libraries

Examples

```
## Not run:
libs <- get_sl_libraries()
head(libs)

## End(Not run)
```

plot.drcmd	<i>Plot results from drcmd object</i>
------------	---------------------------------------

Description

S3 method for plotting results from drcmd object. Plots are available for the following: (1) psuedo outcome regression fit, (2) influence curve distribution, (3) treatment propensity score distribution, and (4) complete-case propensity score distribution. When type='All' (the default), user can view all four plots in succession interactively

Usage

```
## S3 method for class 'drcmd'
plot(x, type = "All", ...)
```

Arguments

x	An object of class drcmd
type	Character denoting type of plot to generate. Must be one of 'All', 'PO', 'IC', 'g_hat', 'r_hat'
...	Additional arguments (unused)

Value

No return value. Called for plotting results from drcmd object

Examples

```
## Not run:
n <- 2500
X <- rnorm(n) ; A <- rbinom(n,1,plogis(X))
Y <- rbinom(n,1,plogis(X-A)) # rnorm(n) + A + X + X^2 + A*X + sin(X) # note: true ATE is 1
Ystar <- Y + rnorm(n)/2 ; R <- rbinom(n,1,plogis(X)) # error-prone outcome measurements

# Make A NA if R==0
A[R==0] <- NA

# Obtain ATE estimates, fitting all nuisance models with ensemble of splines +
# GAMs (save for the pseudo-outcome regression, which is done with XGboost)
drcmd_res <- drcmd(Y,A,covariates,
                  default_learners= c('SL.gam','SL.glm'),
                  po_learners = 'SL.gam')
plot(drcmd_res,type='P0')

## End(Not run)
```

predict.SL.hal9001	<i>Prediction wrapper for the highly-adaptive lasso</i>
--------------------	---

Description

Prediction wrapper for the highly-adaptive lasso (HAL) algorithm implemented in the hal9001 package

Usage

```
## S3 method for class 'SL.hal9001'
predict(object, newdata, ...)
```

Arguments

object	A fitted SL.hal9001 object
newdata	Data frame of covariates for prediction
...	Additional arguments (unused)

Value

Predicted values on new data

Examples

```
## Not run:
n <- 200
X <- data.frame(X1 = rnorm(n))
Y <- X$X1 + rnorm(n)
fit <- SL.hal9001(Y, X, newX = X, family = gaussian(),
  obsWeights = rep(1, n))
preds <- predict(fit$fit, newdata = X)

## End(Not run)
```

print.drcmd	<i>Print drcmd object</i>
-------------	---------------------------

Description

S3 method for printing drcmd objects. Provides concise summary of results, and prints values of optional arguments. Use summary() function for more detailed summary of results.

Usage

```
## S3 method for class 'drcmd'
print(x, ...)
```

Arguments

x	An object of class drcmd
...	Additional arguments (unused)

Value

No return value. Called for printing a concise results summary

Examples

```
set.seed(1)
n <- 200
X <- rnorm(n)
A <- rbinom(n, 1, plogis(X))
Y <- rnorm(n) + A + X
R_ind <- rbinom(n, 1, plogis(X))
Y[R_ind == 0] <- NA
covariates <- data.frame(X = X)

fit <- drcmd(Y, A, covariates, default_learners = "SL.glm", k = 1)
print(fit)
```

SL.hal9001

SuperLearner wrapper for the highly-adaptive lasso

Description

Wrapper for the highly-adaptive lasso (HAL) algorithm implemented in the hal9001 package

Usage

```
SL.hal9001(Y, X, newX, family, obsWeights, ...)
```

Arguments

Y	Outcome variable
X	Matrix of covariates
newX	Matrix of covariates for prediction
family	Family of the outcome variable
obsWeights	Observation weights
...	Additional arguments to pass to the HAL function

Value

A list containing the prediction and the fitted model

Examples

```
## Not run:
n <- 200
X <- data.frame(X1 = rnorm(n))
Y <- X$X1 + rnorm(n)
fit <- SL.hal9001(Y, X, newX = X, family = gaussian(),
  obsWeights = rep(1, n))
head(fit$pred)

## End(Not run)
```

summary.drcmd

Summarize results from drcmd

Description

S3 method for summarizing drcmd results. Provides detailed summary of estimation output, while also providing information on missingness mechanism. Can optionally print out values of user-supplied arguments by setting detail=TRUE

Usage

```
## S3 method for class 'drcmd'
summary(object, detail = FALSE, ...)
```

Arguments

object	An object of class drcmd
detail	Logical. If TRUE, print out values of user-supplied arguments
...	Additional arguments (unused)

Value

No return value. Called for printing a detailed results summary

Examples

```
set.seed(1)
n <- 200
X <- rnorm(n)
A <- rbinom(n, 1, plogis(X))
Y <- rnorm(n) + A + X
R_ind <- rbinom(n, 1, plogis(X))
Y[R_ind == 0] <- NA
covariates <- data.frame(X = X)

fit <- drcmd(Y, A, covariates, default_learners = "SL.glm", k = 1)
summary(fit)
```

tml_updates

Update nuisance functions with TML

Description

Function for updating the nuisance functions m_a and κ through TML

Usage

```
tml_updates(
  idx,
  Y,
  A,
  X,
  R,
  Z,
  m_1_hat,
  m_0_hat,
  g_hat,
  kappa_hat,
  phi_1_hat,
  phi_0_hat,
  varphi_hat,
  maxits = 10
)
```

Arguments

idx	Indices of the training set
Y	Outcome vector
A	Treatment vector
X	Dataframe of covariates
R	Complete case indicator vector
Z	Dataframe containing all variables that are never subject to missingness
m_1_hat	A numeric vector containing the fitted outcome predictions under A=1
m_0_hat	A numeric vector containing the fitted outcome predictions under A=0
g_hat	A numeric vector containing the fitted treatment propensity scores
kappa_hat	A numeric vector containing the fitted values of kappa
phi_1_hat	Estimates of EIC for $E[Y(1)]$
phi_0_hat	Estimates of $E[Y(0)]$
varphi_hat	A list containing estimate of $E[\phi_a Z]$

Value

A list containing updated estimates of the nuisance parameters

Examples

```
## Not run:
# Typically called internally by est_psi_tml() to perform the TML
# targeting step on outcome and missingness nuisance functions.

## End(Not run)
```

trim	<i>Trim vector (for numerical stability)</i>
------	--

Description

Trims values of a vector to avoid numerical instability

Usage

```
trim(x, val = .Machine$double.neg.eps)
```

Arguments

x	A vector of values
val	A small value to add to 0 and subtract from 1

Value

A vector with values trimmed to avoid numerical instability

truncate_g	<i>Truncate treatment propensity scores</i>
------------	---

Description

Truncate propensity scores to interval $[c, 1-c]$

Usage

```
truncate_g(x, cutoff = 0.025)
```

Arguments

x A vector of treatment propensity scores

Value

A vector of treatment propensity scores truncated to interval $[c, 1-c]$

Examples

```
## Not run:
x <- c(0.001, 0.3, 0.5, 0.7, 0.999)
suppressWarnings(truncate_g(x, cutoff = 0.025))

## End(Not run)
```

truncate_r	<i>Truncate complete case propensity scores</i>
------------	---

Description

Truncate propensity scores to interval $[c, 1-c]$

Usage

```
truncate_r(x, cutoff = 0.01)
```

Arguments

x A vector of complete case propensity scores

Value

A vector of complete propensity scores truncated to interval $[c, 1-c]$

Examples

```
## Not run:
x <- c(0.001, 0.3, 0.5, 0.7, 0.999)
suppressWarnings(truncate_r(x, cutoff = 0.01))

## End(Not run)
```